	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez.

Asignatura: Programación orientada a objetos.

Grupo: 07

No de Práctica(s): 10

Integrante(s): 323331229

426060019

323217156

312109239

323195386

321321976

No. de lista o brigada: 6

Semestre: 3ro

Fecha de entrega: 06/11/2026

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción **3**
1.1. Planteamiento del problema 3
1.2. Motivación 3
1.3. Objetivos 3

2. Marco Teórico **4**

3. Desarrollo **5**
3.1. Ejercicio 1 5
3.2. Ejercicio 2 6

4. Resultados **7**
4.1. Resultados del Ejercicio 1 7
4.2. Resultados del Ejercicio 2 8

5. Conclusiones **8**

1. Introducción

1.1. Planteamiento del problema

El problema consiste en el desconocimiento de las excepciones más recurrentes en entornos GUI, se requiere identificar estos errores mediante el uso de IA Generativa. Además necesitamos encontrar mecanismos alternativos para capturar y gestionar aquellas excepciones que un bloque try-catch estándar no puede procesar.

1.2. Motivación

Es indispensable dar solución a este problema debido a que la robustez y la tolerancia a fallos son fundamentales en la experiencia de usuario dentro de cualquier sistema interactivo.

1.3. Objetivos

- **General:** Identificar bloques de código propensos a generar errores y aplicar técnicas adecuadas para el manejo de situaciones excepcionales en tiempo de ejecución.
- **Específicos:**
  - Consultar y clasificar mediante IA Generativa las excepciones más comunes que se producen en una interfaz gráfica (GUI).
  - Identificar los escenarios y tipos de errores que eluden la intercepción de un bloque local try-catch.

2. Marco Teórico

1. **La ley de Pareto:** También conocida como la regla 80/20, establece que el 80 % de las consecuencias proviene del 20 % de las causas. Una máxima en el desarrollo de software dicta que el 80 % del esfuerzo (en tiempo y recursos) produce el 20 % del código. Así mismo, en términos de calidad, el 80 % de las fallas de una aplicación son producidas por el 20 % del código. [1]

2. **Errores sintácticos:** Son todos aquellos errores que se generan por infringir las normas de escritura de un lenguaje: coma, punto y coma, dos puntos, palabras reservadas mal escritas, etc. Normalmente son detectados por el compilador o el intérprete (según el lenguaje de programación utilizado) al procesar el código fuente. [1]

3. **Errores semánticos (o lógicos):** Son errores más sutiles, se producen cuando la sintaxis del código es correcta, pero la semántica o significado no es el que se pretendía. Los compiladores e intérpretes sólo se ocupan de la estructura del código que se escribe y no de su significado, por lo tanto, un error semántico puede hacer que el programa termine de forma anormal, con o sin un mensaje de error. Estos errores se denominan lógicos, ya que, aunque el programa no se bloquea, la lógica que representan contiene un error. [1]

4. **Errores de ejecución:** Son errores que se presentan cuando la aplicación se está ejecutando. Su origen puede ser diverso, se pueden producir por un uso incorrecto del programa por parte del usuario (si ingresa una cadena cuando se espera un número), o se pueden presentar debido a errores de programación (acceder a localidades no reservadas o hacer divisiones entre cero), o debido a algún recurso externo al programa (al acceder a un archivo o al conectarse a algún servidor o cuando se acaba el espacio en la pila (stack) de la memoria). [1]

5. **Excepciones:** El término excepción hace referencia una condición excepcional que cuando ocurre altera el flujo normal del programa en ejecución. Las excepciones son objetos que contienen información del error que se ha producido. Heredan de la clase Exception que a su vez hereda de la clase Throwable. [1]

6. **Excepciones marcadas:** Aquellas cuya captura es obligatoria. [1]

7. **Excepciones no marcadas:** Las excepciones en tiempo de ejecución (RuntimeException y sus subclases). No es obligatorio capturarlas. [1]

8. **Manejo de Excepciones:** Para manejar una excepción se utilizan las palabras reservadas try y catch. El bloque try es utilizado para definir el bloque de código en el cual una excepción pueda ocurrir. El o los bloques catch son utilizados para definir un bloque de código que maneje la excepción. [1]

9. **Throws:** Existen métodos que obligan a ejecutarse dentro de un manejador de excepciones (es decir,son marcadas), debido a que el método define que va a lanzar una excepción. Para indicar que un método puede lanzar una excepción se utiliza la palabra reservada throws seguida de la o las excepciones que puede arrojar dicho método. [1]

3. Desarrollo

3.1. Ejercicio 1

Esta primera actividad consistió en emplear un modelo de lenguaje (LLM) como herramienta de investigación para buscar excepciones más comunes en GUI. Con esto el LLM nos dio la siguiente respuesta:

- **NullPointerException (NPE):** Es, con diferencia, la excepción más común en Swing y JavaFX. Sucede cuando intentas invocar un método o acceder a una propiedad de un componente de la GUI (como un JButton, JLabel o TextField) que no ha sido inicializado (su valor es null).

  Ejemplo: Intentar hacer btnEnviar.setText(.ªceptar"); antes de haber instanciado el botón (btnEnviar = new JButton();).

  Cómo prevenirla: Asegúrate de inicializar todos los componentes gráficos en el constructor o en el método de inicialización antes de usarlos, o verifica si el objeto es nulo antes de invocar métodos sobre él.

- **InvocationTargetException:** Aparece comúnmente al utilizar SwingUtilities.invokeLater() o SwingUtilities.invokeAndWait(). Esta excepción envuelve cualquier otra excepción (como una NullPointerException o NumberFormatException) que haya ocurrido dentro del hilo de despacho de eventos (EDT - Event Dispatch Thread).

  Cómo prevenirla: Debes inspeccionar la causa raíz (getCause()) dentro del bloque catch para identificar el error real que ocurrió en el código que se estaba ejecutando en el hilo gráfico.

- **NumberFormatException:** Al programar GUIs, frecuentemente leemos texto ingresado por el usuario en campos de texto (JTextField o TextField) para convertirlo a números (int, double, etc.). Si el usuario deja el campo vacío o escribe letras en lugar de números, se lanza esta excepción al intentar convertirlo con métodos como Integer.parseInt().

  Ejemplo: int edad = Integer.parseInt(txtEdad.getText()); donde el usuario escribió "Hola".

  Cómo prevenirla: Valida siempre la entrada del usuario antes de realizar la conversión, o utiliza un bloque try-catch para mostrar un mensaje amigable (por ejemplo, mediante un JOptionPane.showMessageDialog) advirtiendo que el valor ingresado no es válido.

- **IllegalArgumentException:** Se lanza cuando le pasas un argumento ilegal o inapropiado a un método de un componente gráfico.

  Ejemplo: Asignar un tamaño, posición o índice fuera de rango, como intentar seleccionar un ítem en un JComboBox o JTable usando un índice negativo o mayor al número de elementos disponibles.

  Cómo prevenirla: Verifica los límites de los índices o los rangos permitidos antes de aplicarlos a tablas, listas o diseños (Layouts).

- **IllegalThreadStateException o problemas de concurrencia (EDT):** Cuándo ocurre: En Java, todos los cambios en la interfaz gráfica deben realizarse exclusivamente en el Hilo de Despacho de Eventos (EDT). Si intentas actualizar la GUI desde un hilo secundario de larga duración (por ejemplo, una descarga de red o una consulta pesada a base de datos), la interfaz puede congelarse, corromperse o lanzar excepciones de actualización de hilos no seguros.

  Cómo prevenirla: En Swing, utiliza SwingUtilities.invokeLater() o la clase SwingWorker para realizar tareas pesadas en segundo plano y actualizar la interfaz solo cuando terminen.En JavaFX, utiliza Platform.runLater() para actualizar los nodos gráficos desde hilos secundarios.

3.2. Ejercicio 2

Para este ejercicio simulamos un problema muy común en la programación con hilos llamado **interbloqueo** (o *deadlock*).

Primero declaramos las clases RecursoA y RecursoB con un atributo tipo String para su nombre. Ambas clases representan dos cosas u objetos que los hilos quieren utilizar:

- **`synchronized`:** Significa que si un hilo entra a usar el recurso, nadie más puede usarlo al mismo tiempo; cualquier otro hilo tendrá que esperaar hasta que el primero termine.
- **(`Thread.sleep`):** Dentro del método `usarRecurso`, hicimos que el hilo se espere 1 segundo antes de continuar. Esto es para darle tiempo al otro hilo de entrar a su propio recurso.
- Después de la pausa, el `RecursoA` intenta entrar a usar a `RecursoB`, y del mismo modo, el `RecursoB` intenta entrar a usar a `RecursoA`.

En la clase Main es donde organizamos todo para que ocurra el problema:

1. Creamos un objeto de `RecursoA` y un objeto de `RecursoB`.
2. **Creamos dos hilos de trabajo:**
   - Al **Hilo 1** le damos la orden: Ve a usar el Recurso A, y desde ahí adentro intenta usar el Recurso B.
   - Al **Hilo 2** le damos la orden contraria: Ve a usar el Recurso B, y desde ahí adentro intenta usar el Recurso A.
3. **Iniciamos los procesos:** Con el método `.start()`, arrancamos los dos hilos casi al mismo tiempo.

Cuando el programa corre, pasa lo siguiente:

1. El **Hilo 1** entra a `RecursoA`.
2. Casi al mismo tiempo, el **Hilo 2** entra a `RecursoB`.

3. Los dos se quedan esperando un segundo por la pausa que programamos.

4. El **Hilo 1** intenta entrar a `RecursoB`, pero se topa con que está siendo utilizado por el Hilo 2, así que se queda parado esperando a que se desocupe.

5. Al mismo tiempo, el **Hilo 2** intenta entrar a `RecursoA`, pero también se topa con que está siendo utilizado por el Hilo 1, y también se queda esperando.

El Hilo 1 no va a soltar el Recurso A hasta que pueda usar el B, pero el Hilo 2 no va a soltar el B hasta que pueda usar el A. Ninguno cede, ninguno avanza y el programa se queda congelado.

Un bloque `try-catch` lanza un aviso formal de error en el sistema (por ejemplo, si intentamos dividir un número entre cero o abrir un archivo que no existe).

En nuestro código:

- Para la computadora, ningún hilo ha hecho nada ilegal, simplemente están esperando su turno para entrar, como el sistema no detecta que algo se rompió, jamás se lanza una excepción.

- El código nunca llega al `catch`: El programa se queda completamente pausado en la línea donde intenta pedir el segundo recurso. Al estar congelado esperando una respuesta que nunca llegará, el flujo de ejecución nunca avanza hacia el bloque `catch`.

Por más que envolvamos todo el código del `Main` dentro de un `try-catch`, la aplicación seguirá quedándose trabada indefinidamente, así que este tipo de fallas lógicas no pueden atraparse con las herramientas tradicionales de manejo de excepciones.

4. Resultados

4.1. Resultados del Ejercicio 1

Se muestran las respuestas obtenidas al consultar a la IA Generativa acerca de las excepciones más comunes de interfaces gráficas de usuario (GUI):



Figura 1: Consulta a el LLM sobre excepciones comunes en GUI.

4.2. Resultados del Ejercicio 2

Al ejecutar el programa en la clase `Main`, se puede apreciar en la terminal cómo ambos hilos inician su ejecución y logran apartar su primer recurso. Sin embargo, en cuanto intentan solicitar el recurso del compañero, el programa se queda completamente congelado:

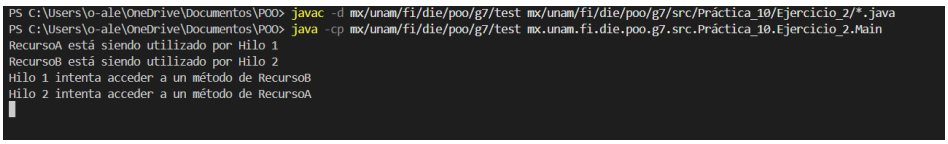


Figura 2: Salida en consola demostrando el congelamiento del programa.

5. Conclusiones

El estudio de errores comunes en entornos GUI demuestra que la robustez de un sistema interactivo depende críticamente de la validación previa de estados y entradas del usuario, evitando así interrupciones abruptas en el hilo de despacho de eventos (EDT). La simulación del interbloqueo (*deadlock*) evidencia que los bloques `try-catch` son inútiles ante fallas de sincronización y concurrencia. Al tratarse de problemas lógicos de bloqueo mutuo y no de excepciones formales de Java, el flujo de ejecución no genera un objeto `Exception`, lo que impide su captura mediante los mecanismos convencionales.

Referencias

[1] Gálvez, J. A. S. (s.f.-a). Excepciones y errores. Unidades de Apoyo Para el Aprendizaje - CUAED - UNAM. https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/3085/mod_resource/content/1/UAPA-Excepciones-Errores/index.html